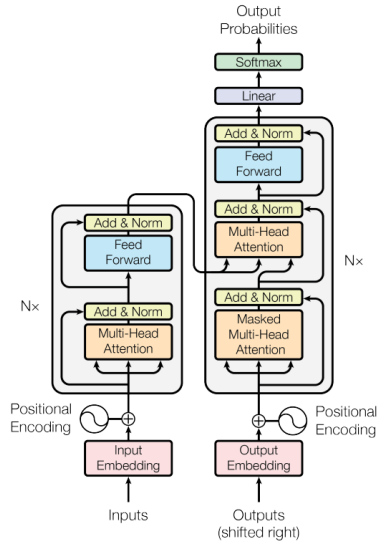
Deep Learning for NLP II

Transformer Recap



Learning goals

- Understand the scope of the course
- Answers to all open question
- Get an impression of the workload

OUTLINE

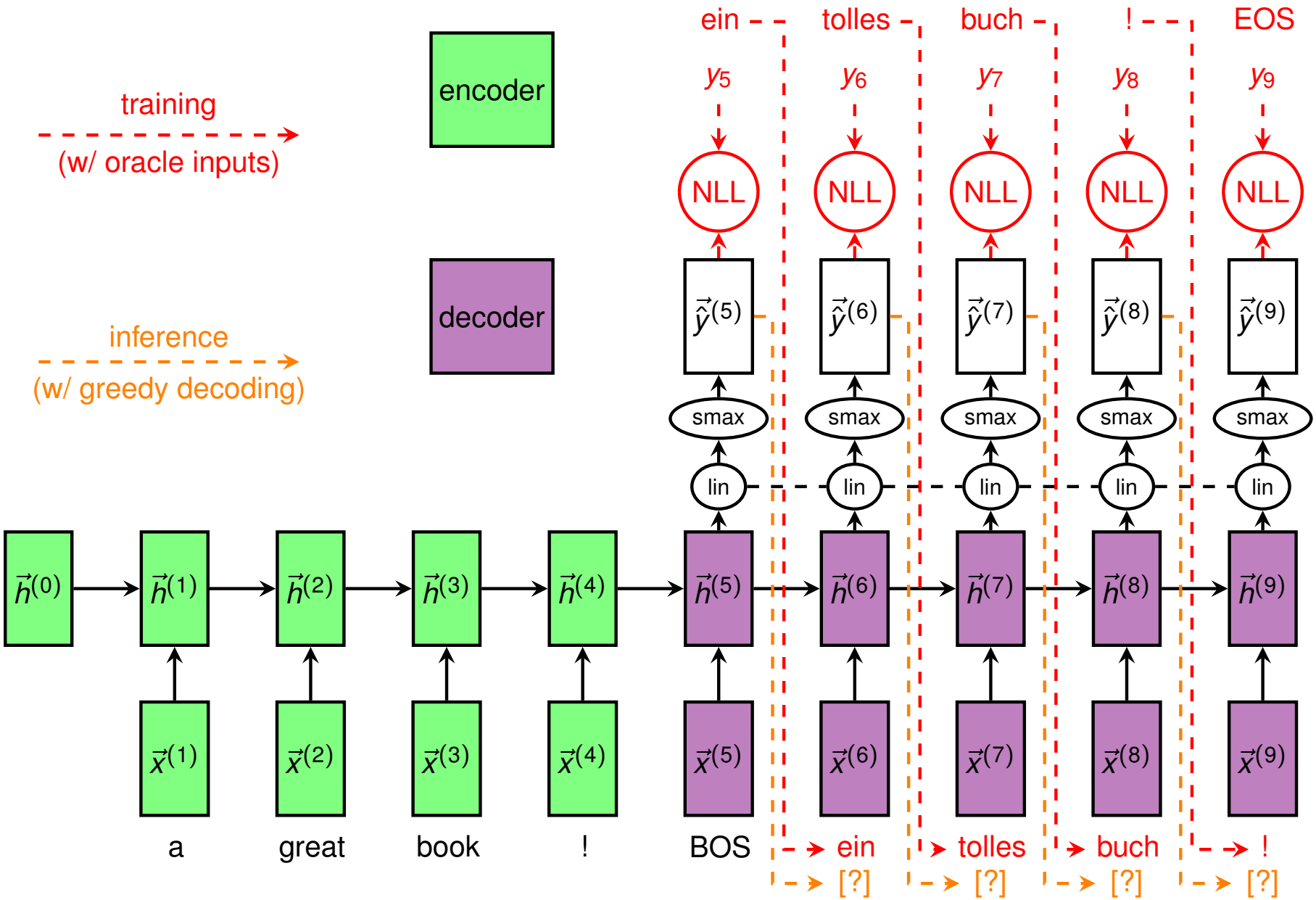
From RNNs to Transformers

Positional Information

Autoregressive Models

Parameter Counts

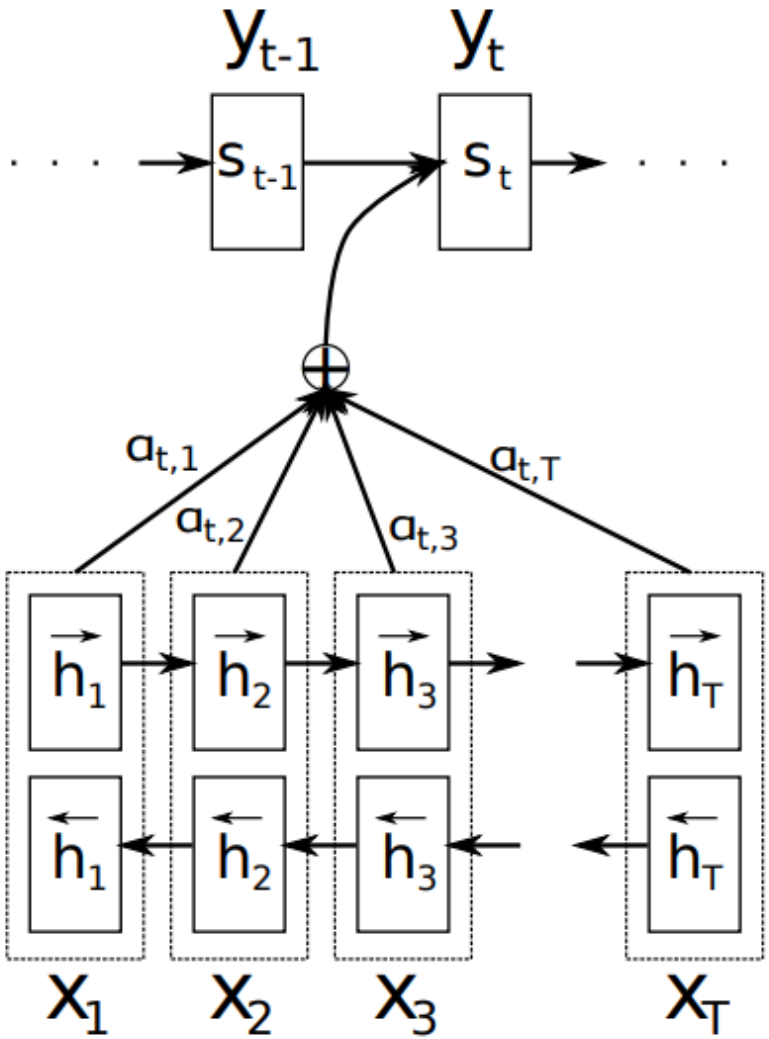
ENCODER-DECODER RNNS



LIMITATIONS OF RNNS

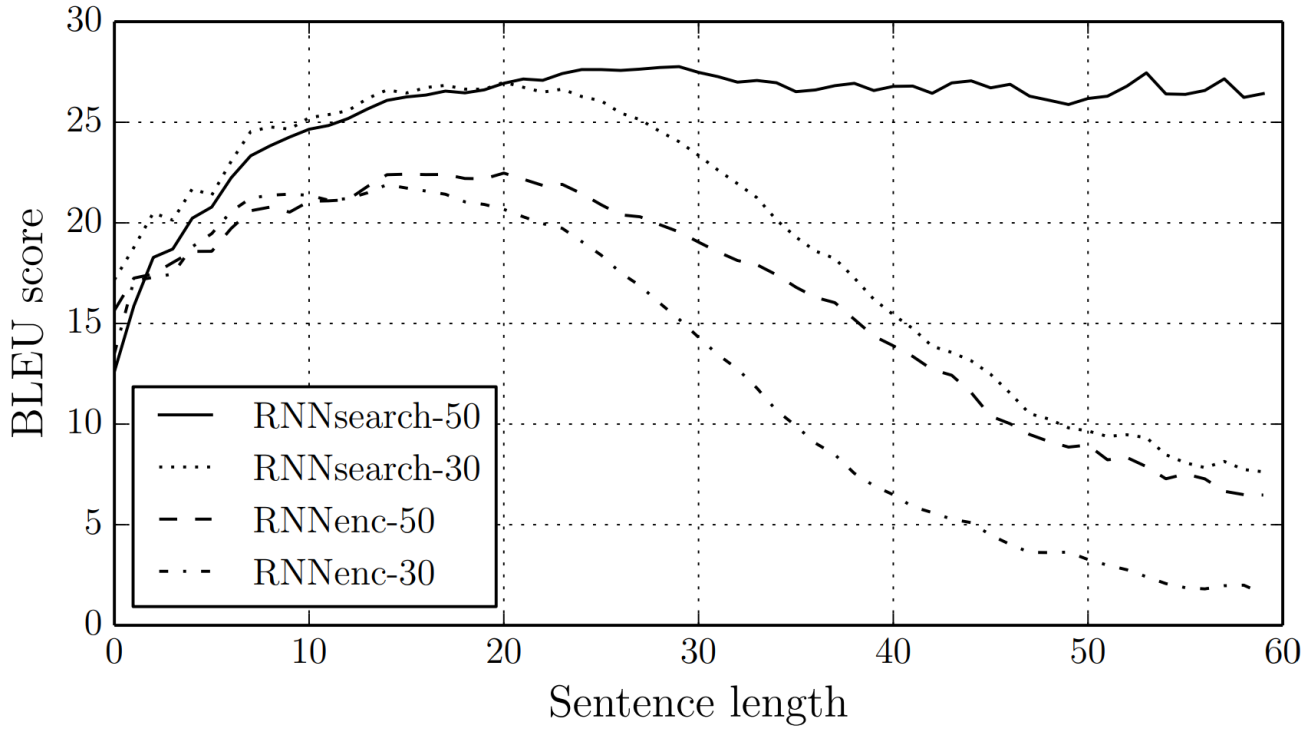
- In an RNN, at a given point in time j, the information about all past inputs $x^{(1)} \ldots x^{(j)}$ is "crammed" into the state vector $\vec{h}^{(j)}$ (and $\vec{c}^{(j)}$ for an LSTM)
- So for long sequences, the state becomes a **bottleneck**
- Especially problematic in encoder-decoder models (e.g., for Machine Translation)
- Solution: Attention ▸ Bahdanau et al., 2015 [?] – an architectural modification of the RNN encoder-decoder that allows the model to "attend to" past encoder states

BAHDANAU ATTENTION



▸ Source: Bahdanau et al., 2015 [**?**]

MODELING LONG-RANGE DEPENDENCIES



▸ Source: Bahdanau et al., 2015 [?]

- RNNenc (classical encoder-decoder); RNNsearch (with Attention)
- BLEU score: Measure for translation quality (higher is better)

ATTENTION: THE BASIC RECIPE (1)

Ingredients:

- One query vector: $\mathbf{q} \in \mathbb{R}^{d_q}$
- J key vectors: $\mathbf{K} \in \mathbb{R}^{J \times d_k}$; $(\vec{k}_1 \ldots \vec{k}_J)$
- J value vectors: $\mathbf{V} \in \mathbb{R}^{J \times d_v}$; $(\vec{v}_1 \ldots \vec{v}_J)$
- Scoring function $a : \mathbb{R}^{d_q} \times \mathbb{R}^{d_k} \rightarrow \mathbb{R}$
  - Maps a query-key pair to a scalar ("score")
  - May be parametrized by parameters θ_a

ATTENTION: THE BASIC RECIPE (2)

- **Step 1**: Apply a to $\vec{q}$ and all keys $\vec{k}_j$ to get scores (one per key):

$$\vec{e} = \begin{bmatrix} e_1 \\ \vdots \\ e_J \end{bmatrix} = \begin{bmatrix} a(\vec{q}, \vec{k}_1; \theta_a) \\ \vdots \\ a(\vec{q}, \vec{k}_J; \theta_a) \end{bmatrix}$$

- **Step 2**: Turn **e** into a probability distribution with the softmax function

$$\alpha_j = \frac{\exp(e_j)}{\sum_{j'=1}^{J} \exp(e_{j'})}$$

  Note that $\sum_j \alpha_j = 1$

ATTENTION: THE BASIC RECIPE (3)

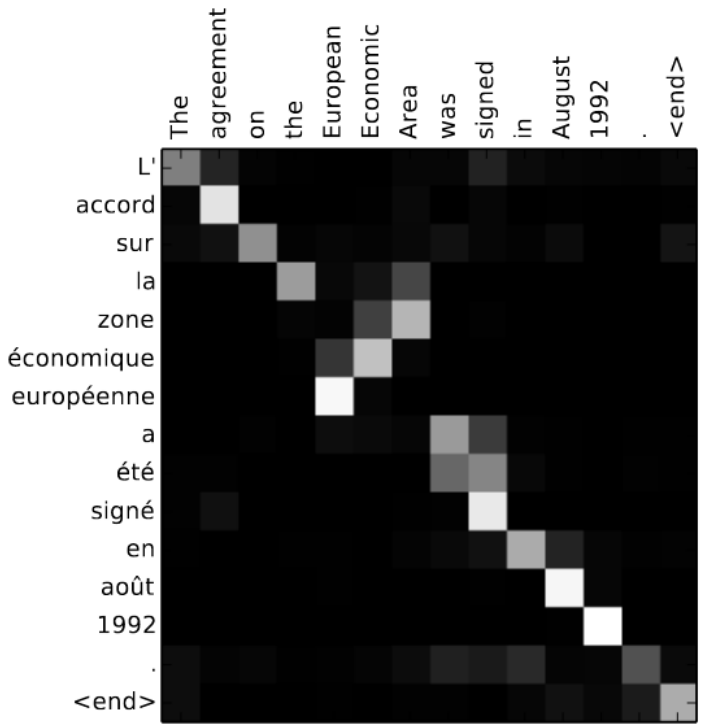
- **Step 3**: α-weighted sum over $\vec{V}$ yields one d_v-dimensional output vector:

$$\vec{o} = \sum_{j=1}^{J} \alpha_j \vec{v}_j$$

Intuition: α_j is how much "attention" the model pays to $\vec{v}_j$ when computing $\vec{o}$.

ATTENTION IN NEURAL NETWORKS

- **Important:** $\vec{q}$, $\vec{k}_j$ and $\vec{v}_j$ vectors of a neural network are a function of the input and trainable parameters
- So the *model* learns which keys are relevant for which queries, based on the training data and loss function



▸ Source: Bahdanau et al., 2015 [**?**]

ATTENTION IN THE TRANSFORMER (1)

- The Bahdanau model is still an RNN, just with attention on top.
- Another architecture that consists of attention only:
  - Transformer: “Attention is all you need” ▸ Vaswani et al., 2017 [**?**]
  - No recurrence, *just* Attention (as the name suggests)
  - Better parallelizable (as opposed to RNNs)
- No (or few) assumptions are baked into the architecture (no notion of which words are neighbors in the sentence, sequentiality, etc.)
- The lack of prior knowledge often means that the Transformer requires more training data than an RNN to achieve a certain performance
- When trained with sufficient data, it usually outperforms them

ATTENTION IN THE TRANSFORMER (2)

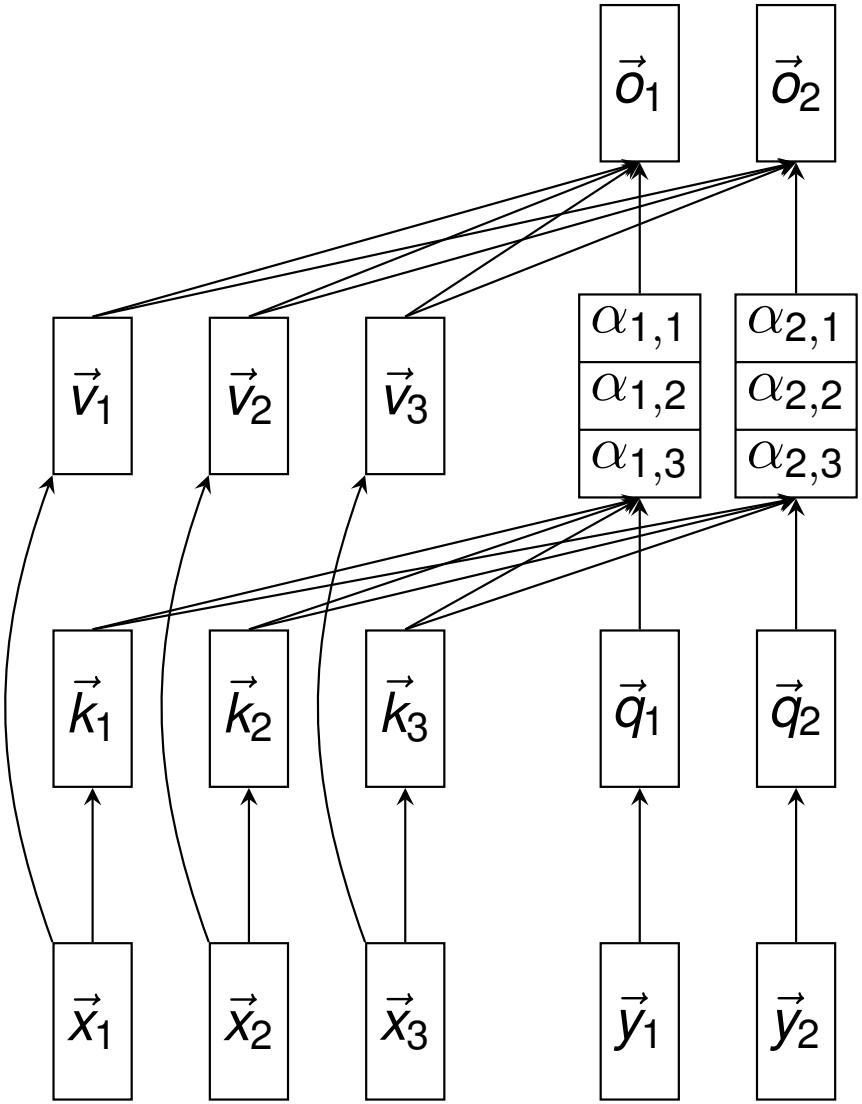
Cross-Attention:

- Let $X = (x_1 \dots x_{J_x})$, $Y = (y_1 \dots y_{J_y})$ be two sequences (e.g., source and target in a sequence-to-sequence problem)
- The query vectors represent tokens in Y and the key/value vectors represent tokens in X ("Y attends to X")

Self-Attention:

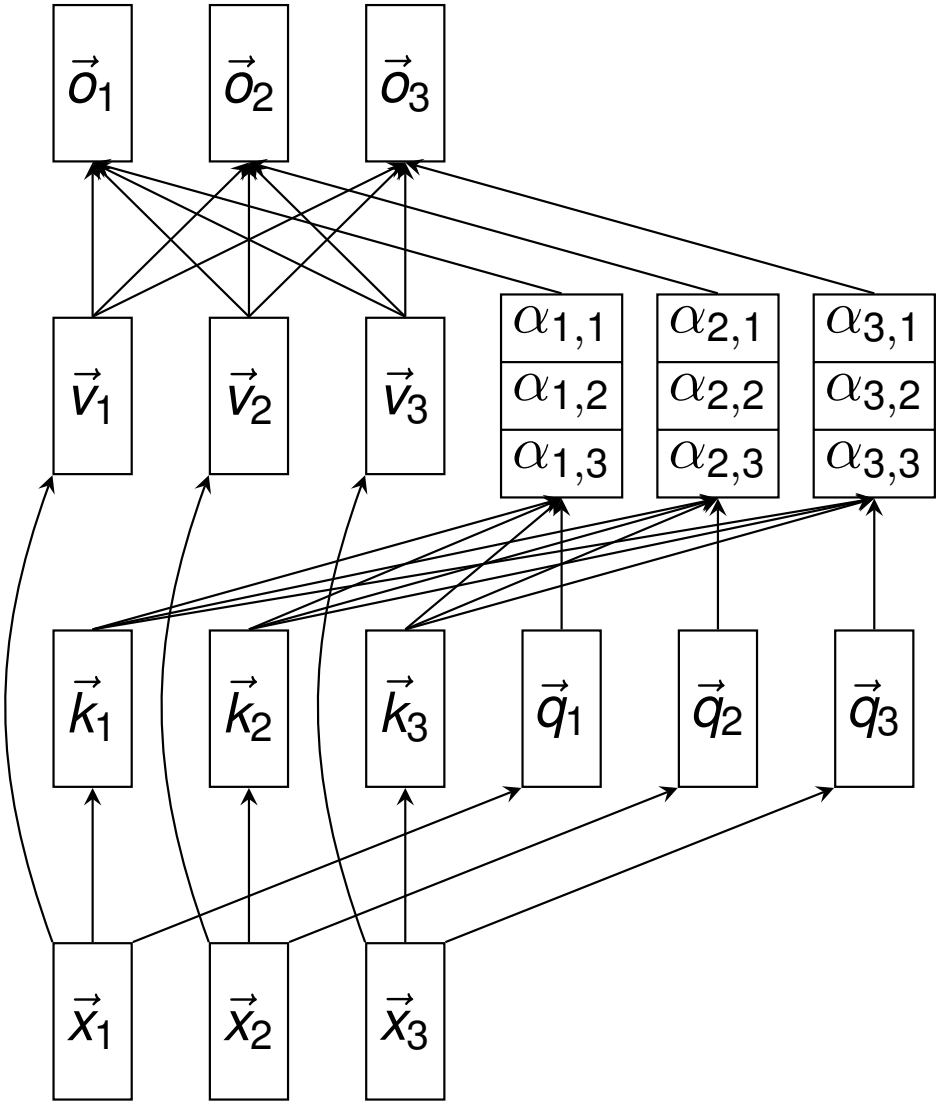
- There is only one sequence $X = (x_1 \dots x_J)$
- The query, key, and value vectors represent tokens in X ("X attends to itself")

CROSS-ATTENTION VS. SELF ATTENTION



Cross-attention

CROSS-ATTENTION VS. SELF ATTENTION



Self-attention

SELF-ATTENTION FORMALIZED

- Let $\vec{X} \in \mathbb{R}^{J_x \times d_x}$ be a representation of X (e.g., stacked word embeddings, or the outputs of a previous layer)
- Let $\theta = \{\vec{W}^{(q)} \in \mathbb{R}^{d_x \times d_q}, \vec{W}^{(k)} \in \mathbb{R}^{d_x \times d_k}, \vec{W}^{(v)} \in \mathbb{R}^{d_x \times d_v}\}$ be trainable weight matrices
- We transform $\vec{X}$ into a matrix of query vectors:

$$\vec{Q} = \vec{X}\vec{W}^{(q)}$$

- And we transform $\vec{X}$ into matrices of key and value vectors:

$$\vec{K} = \vec{X}\vec{W}^{(k)}; \qquad \vec{V} = \vec{X}\vec{W}^{(v)}$$

- Execute Steps 1 – 3 of the basic recipe

... AS A ONE-LINER

$$\vec{O} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{X}; \theta) = \text{softmax}\Big(\frac{(\vec{X}\vec{W}^{(q)})(\vec{X}\vec{W}^{(k)})^T}{\sqrt{d_k}}\Big)(\vec{X}\vec{W}^{(v)})$$

- GPUs like matrix multiplications
  $\rightarrow$ usually a lot faster than RNN!
- But: The memory requirements of $\vec{E}$ and $\vec{A}$ are $\mathcal{O}(J_x^2)$
- Sequence lengths are an important issue.

MULTI-LAYER ATTENTION

- Sequential application of several attention layers, with separate parameters $\{\theta^{(1)} \ldots \theta^{(N)}\}$
- In Transformer: sequential application of Transformer blocks
- There are some additional position-wise layers inside the Transformer block, i.e., $\vec{O}^{(n)}$ undergoes some additional transformations before becoming the input to the next Transformer block $n + 1$

MULTI-HEAD ATTENTION

- Application of several attention layers (“heads”) in parallel
- M sets of parameters $\{\theta^{(1)}, \ldots, \theta^{(M)}\}$, with $\theta^{(m)} = \{\vec{W}^{(m,q)}, \vec{W}^{(m,k)}, \vec{W}^{(m,v)}\}$
- For every head, compute in parallel:

$$\vec{O}^{(m)} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{Y}; \theta^{(m)})$$

- Choice of dimensionality: $d_k = d_v = d_x/M$
- Concatenate all $\vec{O}^{(m)}$ along their last axis; then down-project the concatenation with an additional parameter matrix $\vec{W}^{(o)} \in \mathbb{R}^{Md_v \times d_v}$:

$$\vec{O} = [\vec{O}^{(1)}; \ldots; \vec{O}^{(M)}]\vec{W}^{(o)}$$

OUTLINE

From RNNs to Transformers

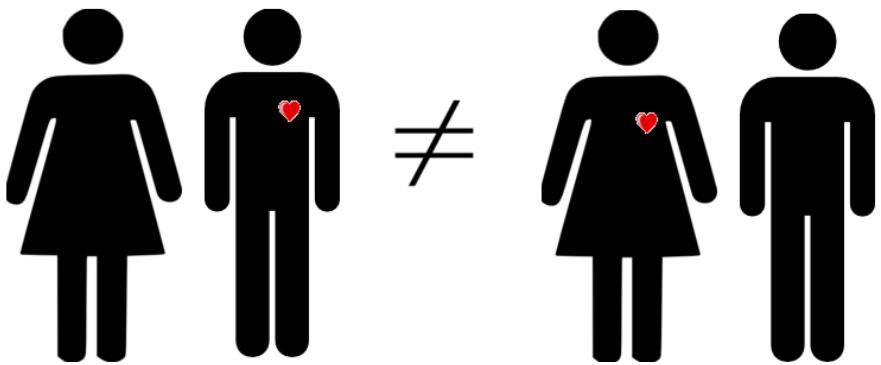
Positional Information

Autoregressive Models

Parameter Counts

CAN SELF-ATTENTION MODEL WORD ORDER?

- Hypothetical model:
  - simple word embedding lookup layer
  - self-attention layer
- Let $X^{(1)}$, $X^{(2)}$ be two sentences of the same length J, which contain the same words, but in a different order
- Example: "john loves mary" vs. "mary loves john"



- But: The representation of mary is identical to that of mary, and the representation of john is identical to that of john

POSITION EMBEDDINGS

- Self-attention computes each output as a *weighted sum of all inputs*
- The weights depend only on content – not on the order of tokens
- Self-attention is therefore **permutation-equivariant**:

$$\text{Attn}(\pi(\vec{x}_1, \ldots, \vec{x}_n)) = \pi\big(\text{Attn}(\vec{x}_1, \ldots, \vec{x}_n)\big) \quad \forall \text{ permutations } \pi$$

- Add to every input word embedding a position embedding $\vec{p} \in \mathbb{R}^d$:
- Input embedding of word "mary" in position j: $\vec{x}_j = \vec{w}_{\mathcal{I}(\text{mary})} + \mathbf{p}_j$

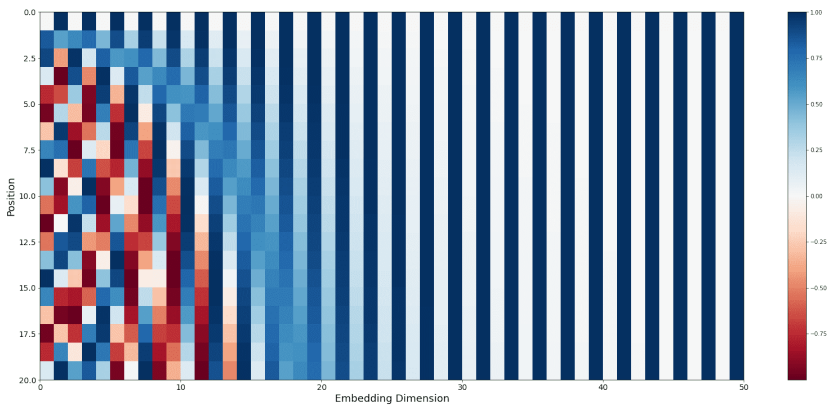
$$\vec{w}_{\mathcal{I}(\text{mary})} + \vec{p}_j \neq \vec{w}_{\mathcal{I}(\text{mary})} + \vec{p}_{j'} \text{ if } j \neq j'$$

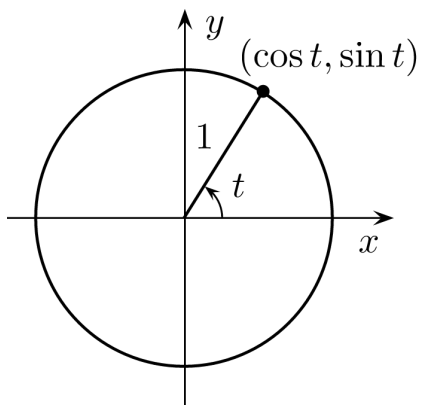
ABSOLUTE POSITIONAL ENCODINGS

- Option 1 (fixed) ▸ Vaswani et al., 2017 [?]
  (Deterministic) Sinusoidal position embeddings:

$$p_{j,i} = \begin{cases} \sin\left(\frac{j}{10000^{\frac{i}{d}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{j}{10000^{\frac{i-1}{d}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

- Dimensions form sine/cosine pairs with same (angular) frequency, which describe rotations on a unit circle:
  - At $i = 0$ and $i = 1$, frequency is 1 radian per token, wavelength $2\pi \approx 6.28$ tokens
  - At $i = d - 2$ and $i = d - 1$, frequency $\approx 10^{-4}$ radians per token, wavelength $\approx 2\pi \cdot 10^4 \approx 62{,}800$ tokens





ABSOLUTE POSITIONAL ENCODINGS

- Option 2 (learned) ▸ Source: Devlin et al., 2019 [**?**]
  Trainable position embeddings: $\vec{P} \in \mathbb{R}^{J^{\max} \times d}$
  (Disadvantage: Cannot deal with sentences longer than $J^{\max}$)

RELATIVE POSITIONAL ENCODINGS

- Key idea: inject relative offset $(i - j)$ directly into the attention computation ▸ Shaw et al. (2018) [?]
- Learnable per-offset vectors $\vec{a}^K_{i-j}$ and $\vec{a}^V_{i-j}$ augment keys and values:

$$e_{i,j} = \frac{(\vec{x}_i \mathbf{W}^Q)(\vec{x}_j \mathbf{W}^K + \vec{a}^K_{i-j})^\top}{\sqrt{d_k}}, \qquad \vec{o}_i = \sum_j \alpha_{i,j}(\vec{x}_j \mathbf{W}^V + \vec{a}^V_{i-j})$$

- The model now conditions *directly* on relative distance when computing each attention weight
- Offsets are clipped to $[-k, k]$ to keep the learnable table finite: $\vec{a}^K_{\max(-k, \min(k, i-j))}$
- Requires $O(n^2)$ additional terms – memory overhead for long sequences

ATTENTION WITH LINEAR BIASES

- Completely removes position embeddings; instead adds a *fixed* scalar bias to each attention logit ▸ ALiBi (Press et al., 2022) [?]
- The bias penalizes attention to distant tokens proportionally to their distance:

$$e_{i,j} = \frac{\vec{q}_i^{\top} \vec{k}_j}{\sqrt{d_k}} - m_h \cdot |i - j|$$

- Slope m_h is head-specific and *fixed* (geometric sequence: $\frac{1}{2^1}, \frac{1}{2^2}, \ldots, \frac{1}{2^H}$)
- No position vectors added to inputs; no extra parameters to train
- Strong length-extrapolation: trained on 1024 tokens, generalizes well to 2048+
- Used in MPT, BLOOM, and other large open-weight LLMs

ROTARY POSITIONAL EMBEDDINGS

- RoPE encodes position by *rotating* query/key vectors in consecutive 2D subspaces ▸ Su et al. (2021) [**?**]
- For each pair of dimensions $(2m, 2m+1)$, apply a rotation by angle $j \cdot \theta_m$:

$$\theta_m = \text{base}^{-2m/d}, \quad \text{base} = 10000$$

$$\mathrm{RoPE}(\vec{q}, j) = \mathbf{R}(j)\vec{q},$$

$$\mathbf{R}(j) = \mathrm{diag}\left(\begin{bmatrix} \cos j\theta_0 & -\sin j\theta_0 \\ \sin j\theta_0 & \cos j\theta_0 \end{bmatrix}, \ \ldots, \ \begin{bmatrix} \cos j\theta_{d/2-1} & -\sin j\theta_{d/2-1} \\ \sin j\theta_{d/2-1} & \cos j\theta_{d/2-1} \end{bmatrix} \right)$$

- Applied to $\vec{q}$ and $\vec{k}$ only – no change to values or embeddings

OUTLINE

From RNNs to Transformers

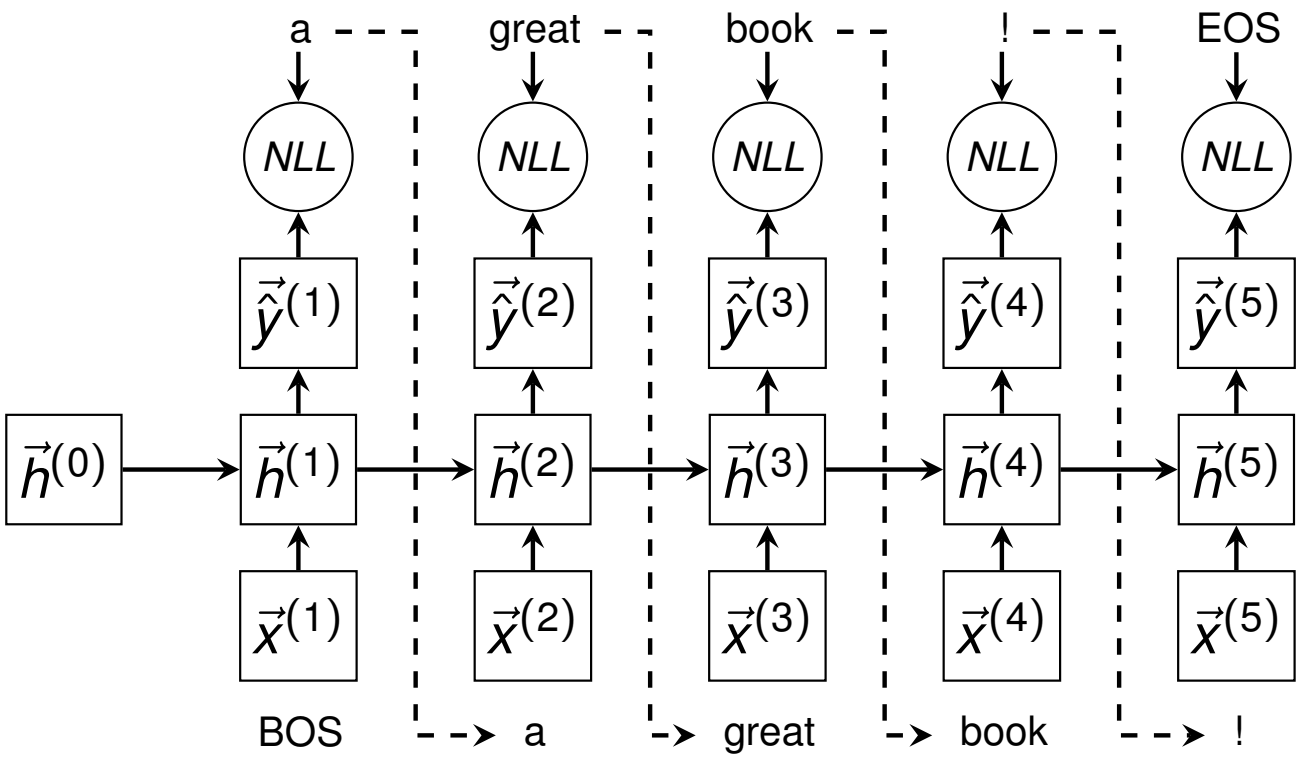
Positional Information

Autoregressive Models

Parameter Counts

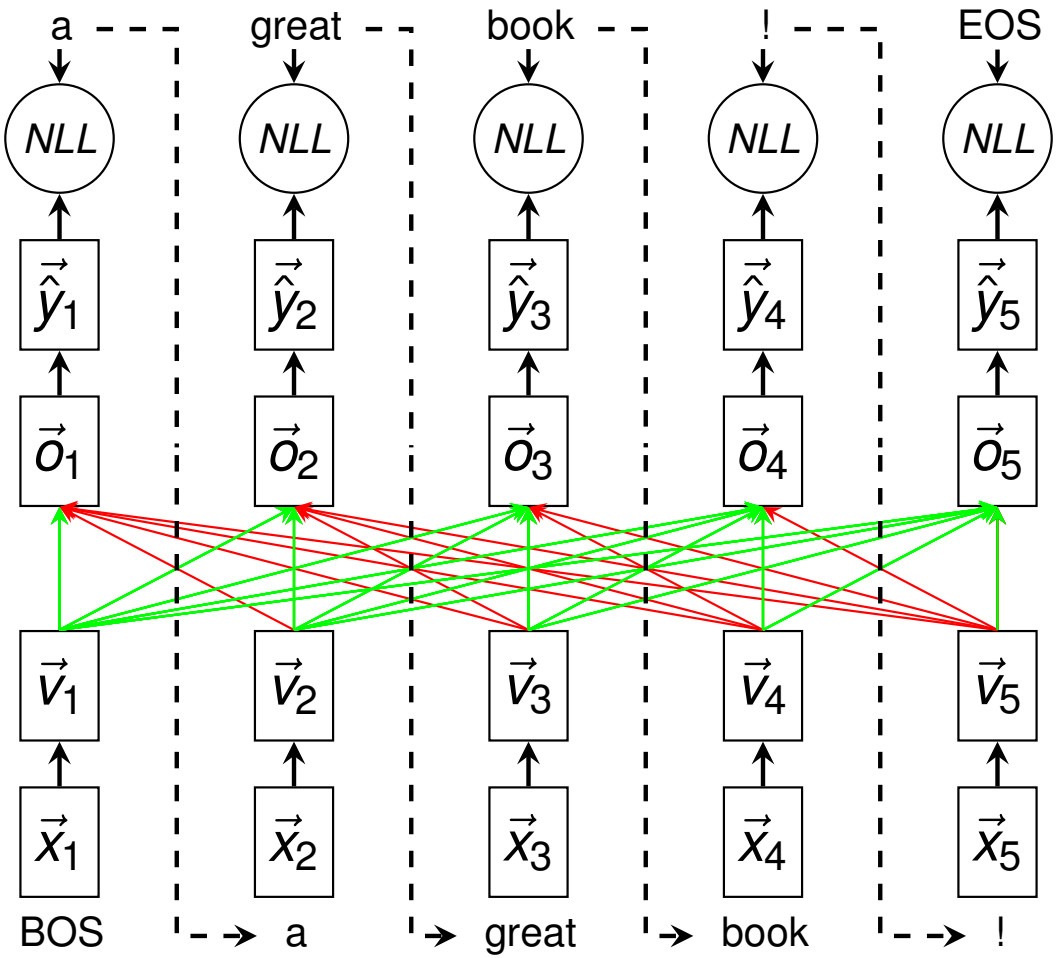
RNNS FOR AUTOREGRESSIVE LM & DECODING

- In autoregressive language modeling, or in the decoder of a sequence-to-sequence model, the task is to always predict the next word
- In an RNN, a given state $\vec{h}^{(j)}$ depends on past inputs $x^{(1)} \ldots x^{(j)}$
- Thus, the RNN is unable to "cheat":



SELF-ATTENTION FOR AR LM & DECODING

- With attention, all $\vec{o}_j$ depend on all $\vec{v}_{j'}$ (and by extension, all $\vec{x}_{j'}$).
- This means that the model can easily cheat by looking at future words (red connections)



PARALLELIZED MASKED SELF-ATTENTION

- Step 1: Calculate $\vec{E}$ like we usually would
- Step 1B:

$$\vec{E}^{\text{masked}} = \vec{E} \odot \vec{M} + \infty\vec{M} - \infty; \qquad m_{j,j'} = \begin{cases} 1 & \text{if } j' \leq j \\ 0 & \text{otherwise} \end{cases}$$

- Example:

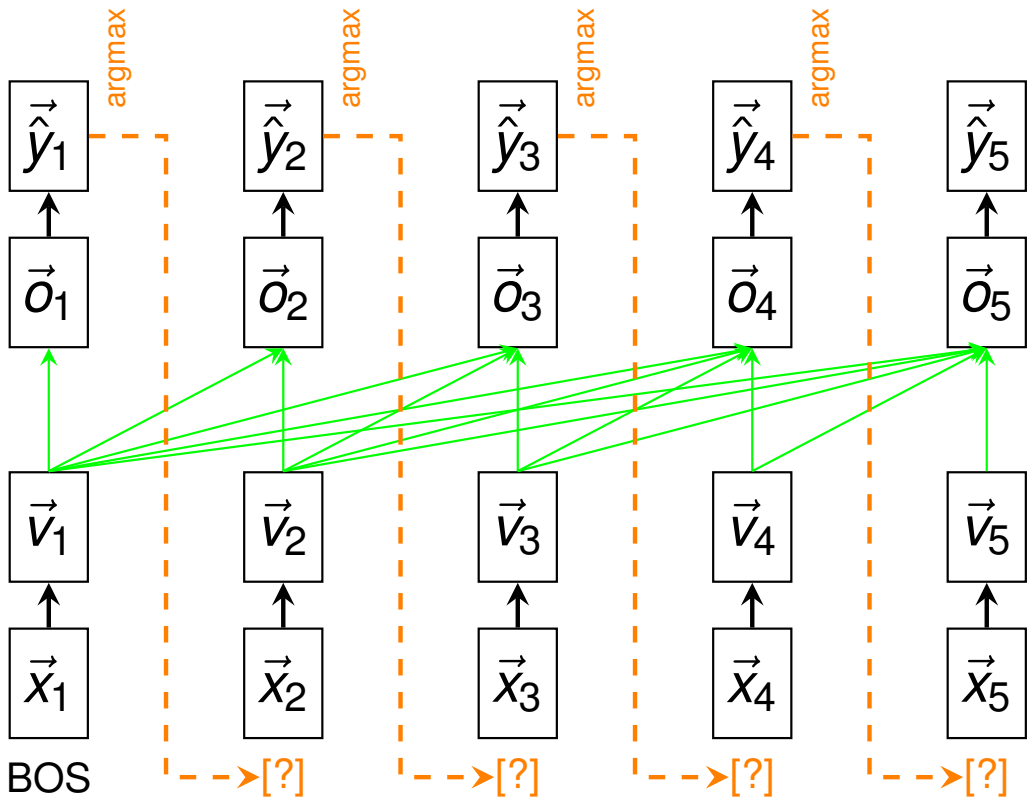
$$\vec{E} = \begin{bmatrix} e_{1,1} & e_{1,2} & e_{1,3} \\ e_{2,1} & e_{2,2} & e_{2,3} \\ e_{3,1} & e_{3,2} & e_{3,3} \end{bmatrix}; \qquad \vec{M} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\vec{E}^{\text{masked}} = \begin{bmatrix} e_{1,1} & -\infty & -\infty \\ e_{2,1} & e_{2,2} & -\infty \\ e_{3,1} & e_{3,2} & e_{3,3} \end{bmatrix}; \qquad \vec{A}^{\text{masked}} = \begin{bmatrix} 1 & 0 & 0 \\ \alpha_{2,1} & \alpha_{2,2} & 0 \\ \alpha_{3,1} & \alpha_{3,2} & \alpha_{3,3} \end{bmatrix}$$

$$\vec{o}_1 = \vec{v}_1; \qquad \vec{o}_2 = \alpha_{2,1}\vec{v}_1 + \alpha_{2,2}\vec{v}_2; \qquad \vec{o}_3 = \alpha_{3,1}\vec{v}_1 + \alpha_{3,2}\vec{v}_2 + \alpha_{3,3}\vec{v}_3$$

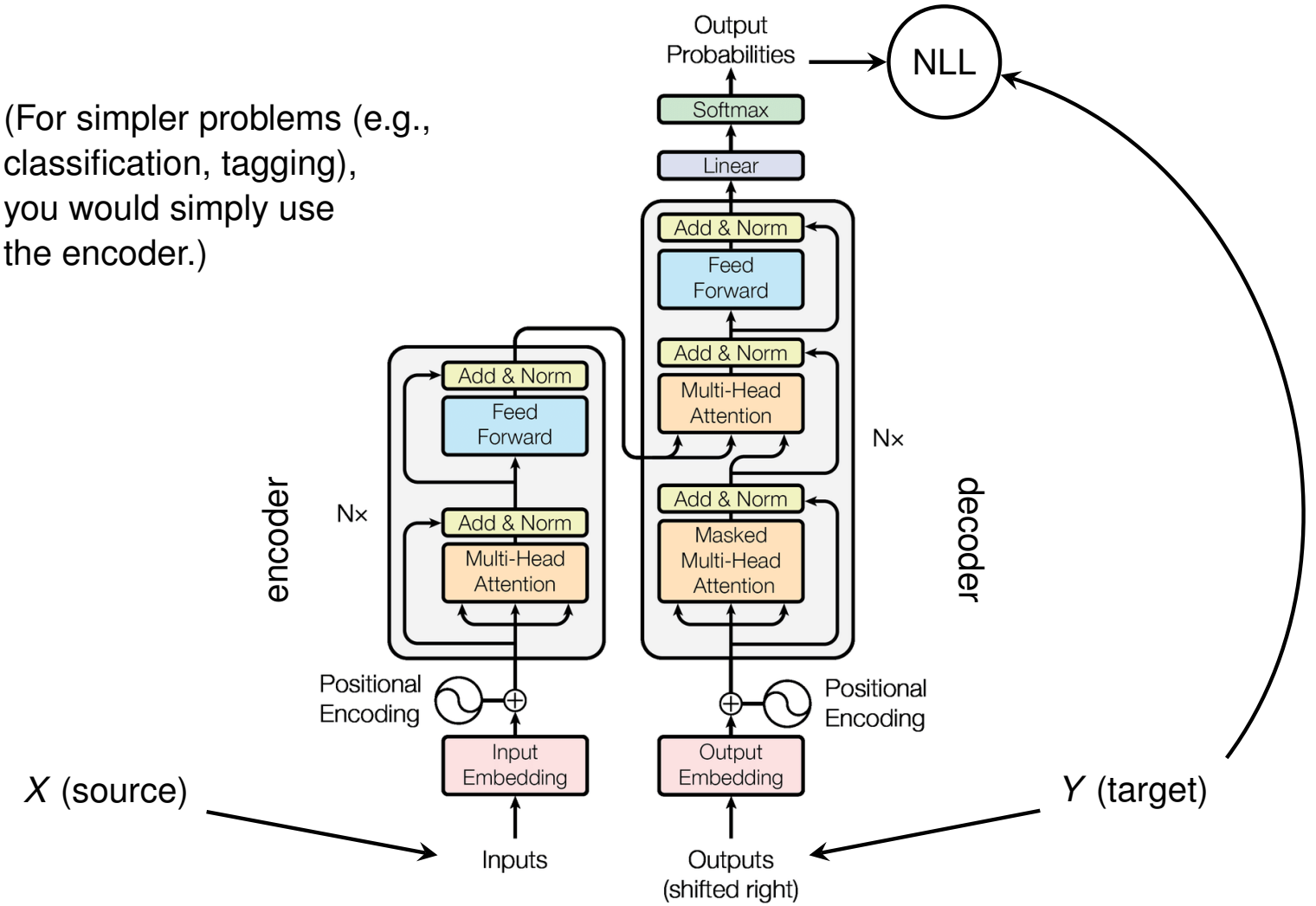
AR TRANSFORMER AT INFERENCE TIME

- During training (targets known): Use parallelized masked attention
- At inference time (targets unknown): Decode prediction in a loop
- Slower, but at least we don't have to worry about masking anymore



THE TRANSFORMER ARCHITECTURE

(For simpler problems (e.g., classification, tagging), you would simply use the encoder.)



▸ Source: Vaswani et al., 2017 [?]

WHY DO WE CARE ABOUT THE PARAMETER COUNT?

- Being able to derive the parameter count of a model requires good knowledge of the math behind the model architecture
- So by deriving the parameter count, we learn how the model works under the hood
- Since models just stack various building blocks on top of each other, we can reuse the derived parameter count for all sorts of models
- We get an idea about how the parameters scale with increasing model size

OUTLINE

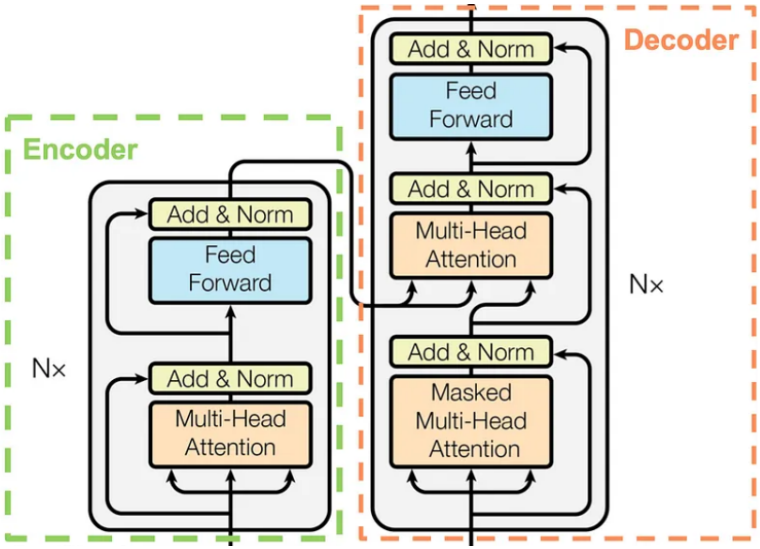
From RNNs to Transformers

Positional Information

Autoregressive Models

Parameter Counts

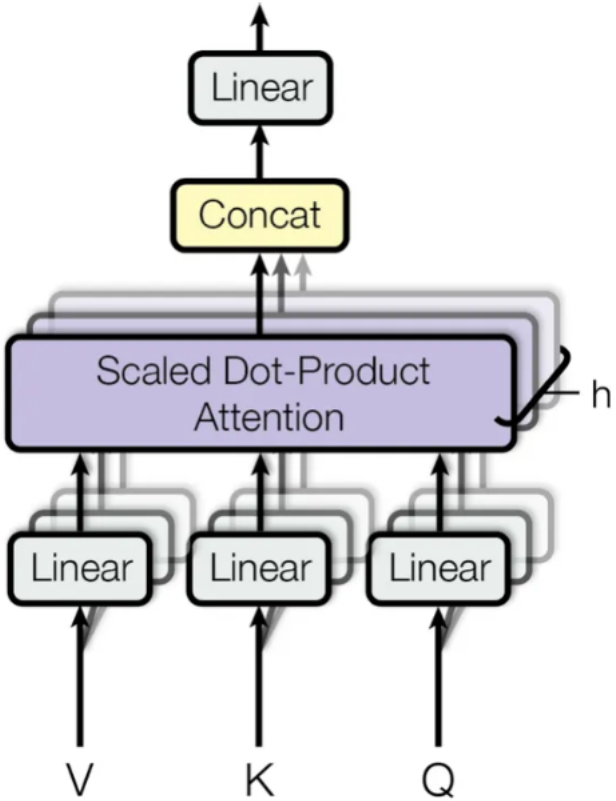
OVERVIEW OVER PARAMETER SOURCES



- **Encoder**
  - Multi-Head Attention
  - 2x Layernorm
  - Feed Forward Network
- **Decoder**
  - 2x Multi-Head Attention
  - 3x Layernorm
  - Feed Forward Network

So all we have to do is to derive the parameter count of each of those components and then sum them up! ▸ Vaswani et al., 2017

MULTI HEAD ATTENTION (1)



- The input token embeddings have shape d_{model} (embedding dimension)
- Project input embeddings into Query, Keys and Values with matrices $\boldsymbol{W_Q}$, $\boldsymbol{W_K}$, $\boldsymbol{W_V}$ for each head (so we have n_{head} (h in the image) of each Matrix)
- Project concatenated output of the Scaled Dot-Product Attention back to d_{model} with $\boldsymbol{W_O}$

To get to the total parameter count, we simply have to calculate the number of parameters in all of those Matrices! (for now, we also include the bias terms of the linear projections)

MULTI HEAD ATTENTION (2)

- **Project Input Sequence into Query, Key and Value:**
  - Input shape: $seq_len \times d_{model}$ (seq_len is the number of tokens, where each has the embedding dimension of d_{model})
  - For MHA we project d_{model} to d_{qkv}, with $d_{qkv} = d_{model} / n_{head}$
  - We split up d_{model} into n_{head} heads and need $\boldsymbol{W_Q}$, $\boldsymbol{W_K}$, $\boldsymbol{W_V}$ for each head
  - Each Matrix has shape $d_{model} \times d_{qkv}$

- **Project concatenated output to the MHA output:**
  - We concatenate the n_{head} outputs, where each has shape d_{qkv} and end up with a shape of d_{model} again
  - We then project d_{model} to d_{model} with $\boldsymbol{W_O}$
  - $\boldsymbol{W_O}$ thus has a shape of $d_{model} \times d_{model}$

MULTI HEAD ATTENTION (3)

Now we put both parts together:

$$
\begin{aligned}
N_{attention} &= \underbrace{(d_{model} \times d_{model} + d_{model})}_{W_O} \\
&\quad + \underbrace{(d_{model} \times d_{qkv} + d_{qkv})}_{W_{Q,K,V}} \times \overbrace{n_{heads} \times 3}^{\text{3 matrices for each head}} \\
&= (d_{model} \times d_{model} + d_{model}) + (d_{model} \times d_{model} + d_{model}) \times 3 \\
&= \underbrace{(d_{model}^2 + d_{model}) \times 4}_{\text{The exact formula}} \\
&\approx \underbrace{4 \cdot d_{model}^2}_{\text{Approximation}}
\end{aligned}
$$

FEED FORWARD NETWORK

- The FFN of the Transformer uses two fully connected layers
- Input and output size is both d_{model}
- The hidden layer size is $4 \times d_{model}$ (*This is a decision by the authors of the paper!)*
- Our weight matrices W_1 and W_2 therefore have shape $d_{model} \times 4 \cdot d_{model}$ and $4 \cdot d_{model} \times d_{model}$ respectively

$$
\begin{aligned}
N_{FFN} &= (d_{model} \times 4 \cdot d_{model} + 4 \cdot d_{model}) + (4 \cdot d_{model} \times d_{model} + d_{model}) \\
&= 8 \cdot d_{model}^2 + 5 \cdot d_{model} \\
&\approx 8 \cdot d_{model}^2
\end{aligned}
$$

LAYER NORM

- We estimate μ and σ for each layer (*H denotes the number of hidden neurons in a layer*):

$$\mu = \frac{1}{N}\sum_{i=1}^{H} a_i$$

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{H}(a_i - \mu)^2}$$

- For rescaling the normalizations we need a β and γ parameter for each neuron in the layer
- Since our layer has d_{model} neurons we need $2 \cdot d_{model}$ parameters for the Layer Norm operations

$$N_{Layernorm} = 2 \cdot d_{model}$$

EXACT PARAMETER COUNT

All we have to do now is to sum up the Parameter counts of each component for both the Encoder and Decoder!

$$
\begin{aligned}
N_{Encoder} &= 1 \cdot N_{attention} + 1 \cdot N_{FFN} + 2 \cdot N_{Layernorm} \\
&= 4 \cdot (d_{model}^2 + d_{model}) + 8 \cdot d_{model}^2 + 5 \cdot d_{model} + 2 \cdot (2 \cdot d_{model}) \\
&= 12 \cdot d_{model}^2 + 13 \cdot d_{model}
\end{aligned}
$$

$$
\begin{aligned}
N_{Decoder} &= 2 \cdot N_{attention} + 1 \cdot N_{FFN} + 3 \cdot N_{Layernorm} \\
&= 2 \cdot 4 \cdot (d_{model}^2 + d_{model}) + 8 \cdot d_{model}^2 + 5 \cdot d_{model} + 3 \cdot (2 \cdot d_{model}) \\
&= 16 \cdot d_{model}^2 + 19 \cdot d_{model}
\end{aligned}
$$

$$
\begin{aligned}
N_{Total} &= 12 \cdot d_{model}^2 + 13 \cdot d_{model} + 16 \cdot d_{model}^2 + 19 \cdot d_{model} \\
&= 28 \cdot d_{model}^2 + 29 \cdot d_{model}
\end{aligned}
$$

APPROXIMATE PARAMETER COUNT

Since the parameter count scales quadratically with d_{model} we can approximate it by omitting the layernorm parameters and the bias terms!

$$N_{Encoder} = 1 \cdot N_{attention} + 1 \cdot N_{FFN} = 4 \cdot d_{model}^2 + 8 \cdot d_{model}^2 = 12 \cdot d_{model}^2$$

$$N_{Decoder} = 2 \cdot N_{attention} + 1 \cdot N_{FFN} = 2 \cdot 4 \cdot d_{model}^2 + 8 \cdot d_{model}^2 = 16 \cdot d_{model}^2$$

$$N_{Total} = 12 \cdot d_{model}^2 + 16 \cdot d_{model}^2 = 28 \cdot d_{model}^2$$

This is the approximate parameter count for one layer!

CALCULATE PARAMETER COUNT

Now we can calculate the parameter count of the vanilla transformer

- From the paper we know $n_{Layers} = 6$ for both Encoder and Decoder and $d_{model} = 512$
- Let's first use the exact formula:

$$\begin{aligned} N_{Params} &= 6 \cdot (12 \cdot 512^2 + 13 \cdot 512) + 6 \cdot (16 \cdot 512^2 + 19 \cdot 512) \\ &= 44,138,496 \end{aligned}$$

- And now with the approximation:

$$\begin{aligned} N_{Params} &= 6 \cdot (12 \cdot 512^2) + 6 \cdot (16 \cdot 512^2) \\ &= 44,040,192 \end{aligned}$$

The approximation is really good, which is why we can ignore the bias and layernorm parameters!

EMBEDDING PARAMETERS

We also have to consider the parameters of the embeddings:

- Let M be the maximum sequence length and V the size of the vocabulary
- We have $M \times d_{model}$ positional embedding parameters, which are fixed and not learned
- And $V \times d_{model}$ token embeddings parameters, which are learned
- The original transformer has $V = 37000$ tokens:

$$N_{Embedding} = 37000 \cdot 512 = 18,944,000$$

- Adding those to the previous count yields:

$$N_{Final} = 18,944,000 + 44,040,192 = 62,984,192$$

REFERENCES